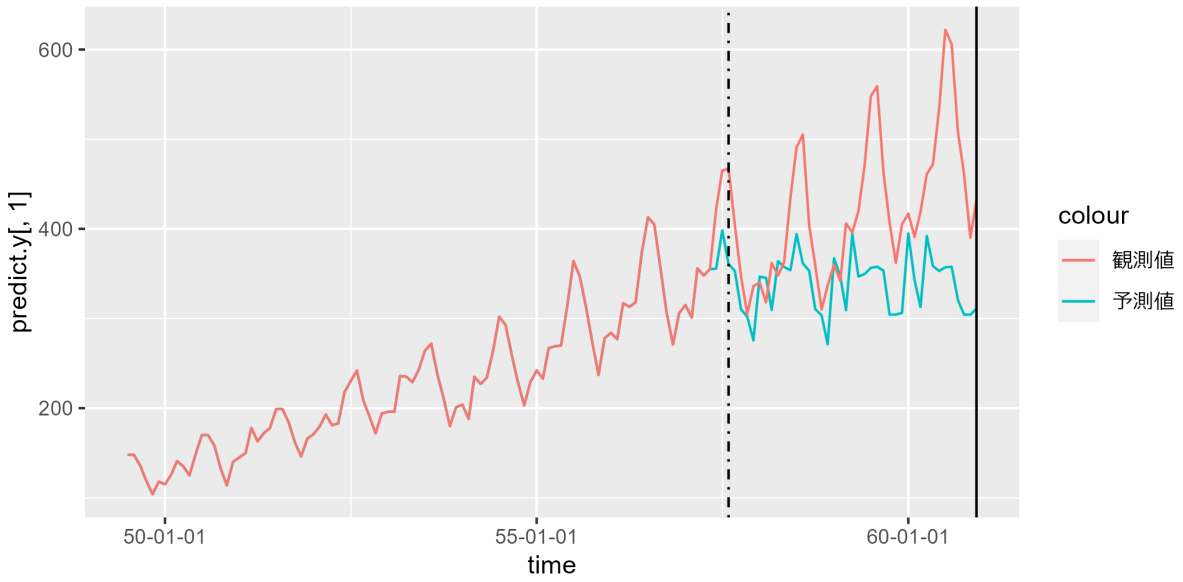
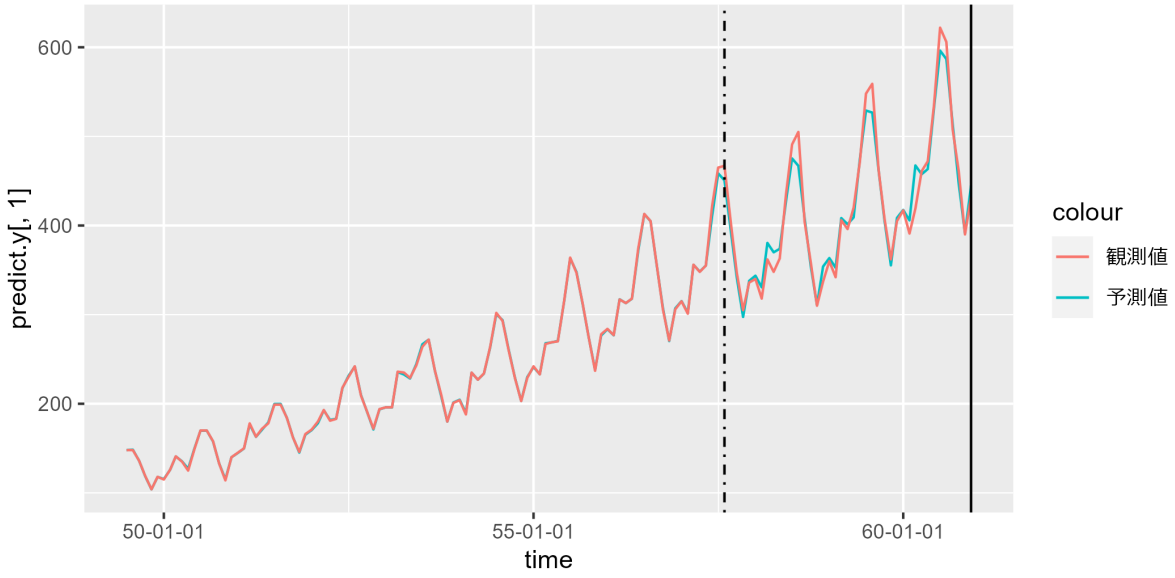
# Application of xgboost to time series forecasting by taking advantage of its powerful forecasting performance

The **xgboost** has the best forecasting performance among **non-deep learning** methods. However, it works well for interpolation problems and regression, but not for future forecasting of time series data that requires extrapolation. I think it is difficult to avoid this tendency even if we add possible explanatory variables.
Possible explanatory variables include lags of a day or several days from the data, month, day, day of the week, holidays, and so on. In fact, the increase or decrease in data values due to these factors is quite possible and can serve as explanatory variables. However, even if you do this, you will not be able to capture the trend.




This is especially true for time series data where there is a trend. The main reason is simple. The main reason is simple: they have very different basic statistics (e.g., mean) from the trained interval, information that is not acquired by training at all.

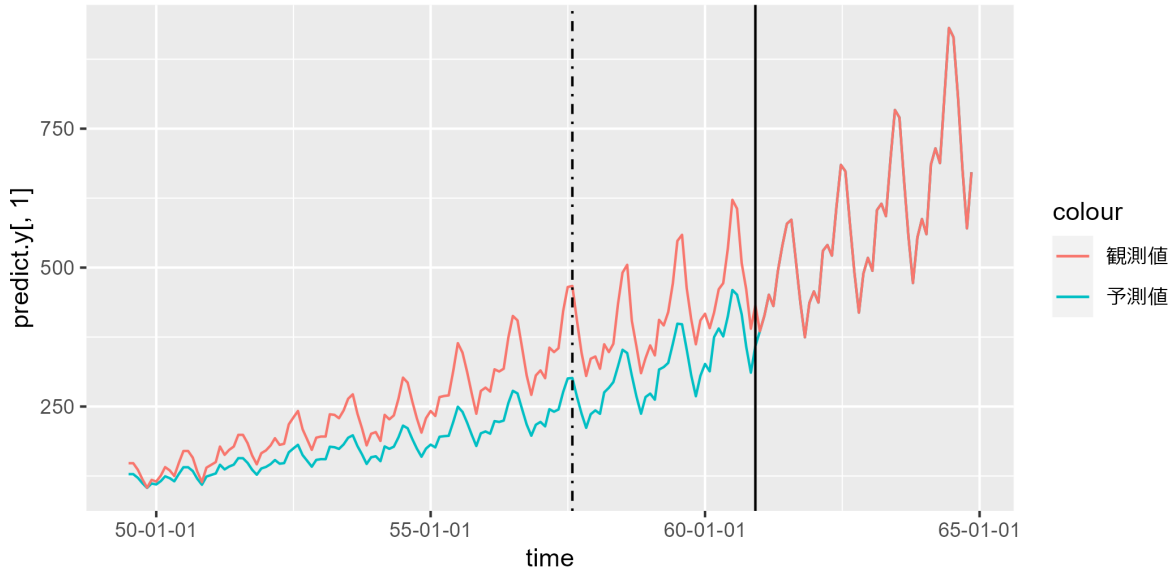
Normally, when we focus on stationarity, that is, the mean and variance of the data, we want it to show roughly the same trend at all points in time. Therefore, by taking the difference and logarithm of the data, we can make the data stationary, create a prediction model, and then reverse the difference and logarithm transformation of the prediction results.

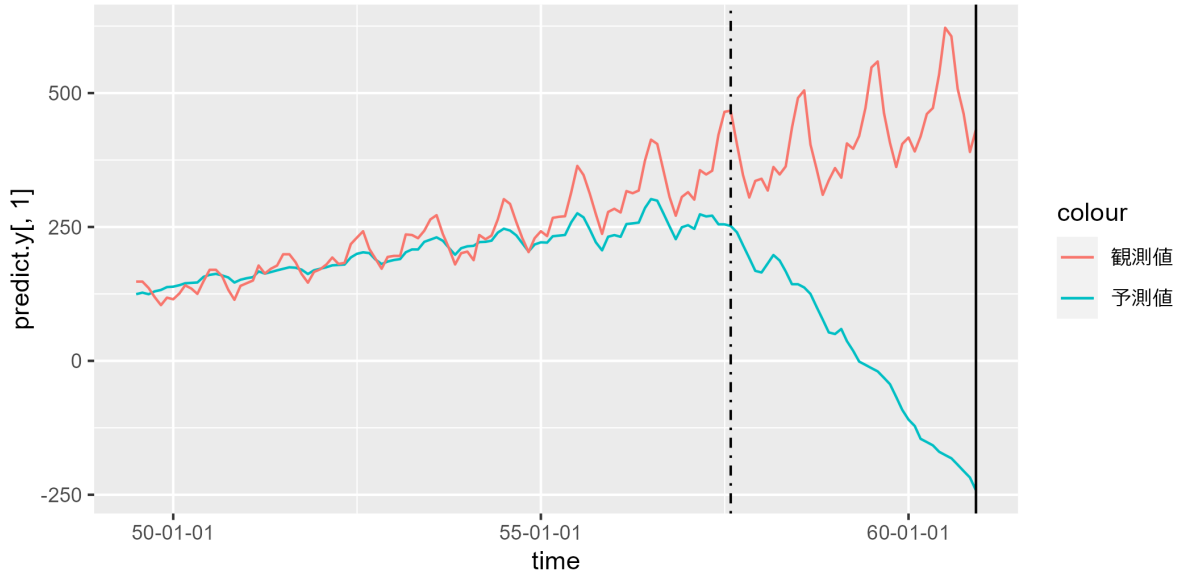



For data that has been differenced and log-transformed in this way, **xgboost** can be a seemingly good predictive model.

However, this is only the case if the fact that the data is stationary by differencing can be observed at any point in time.

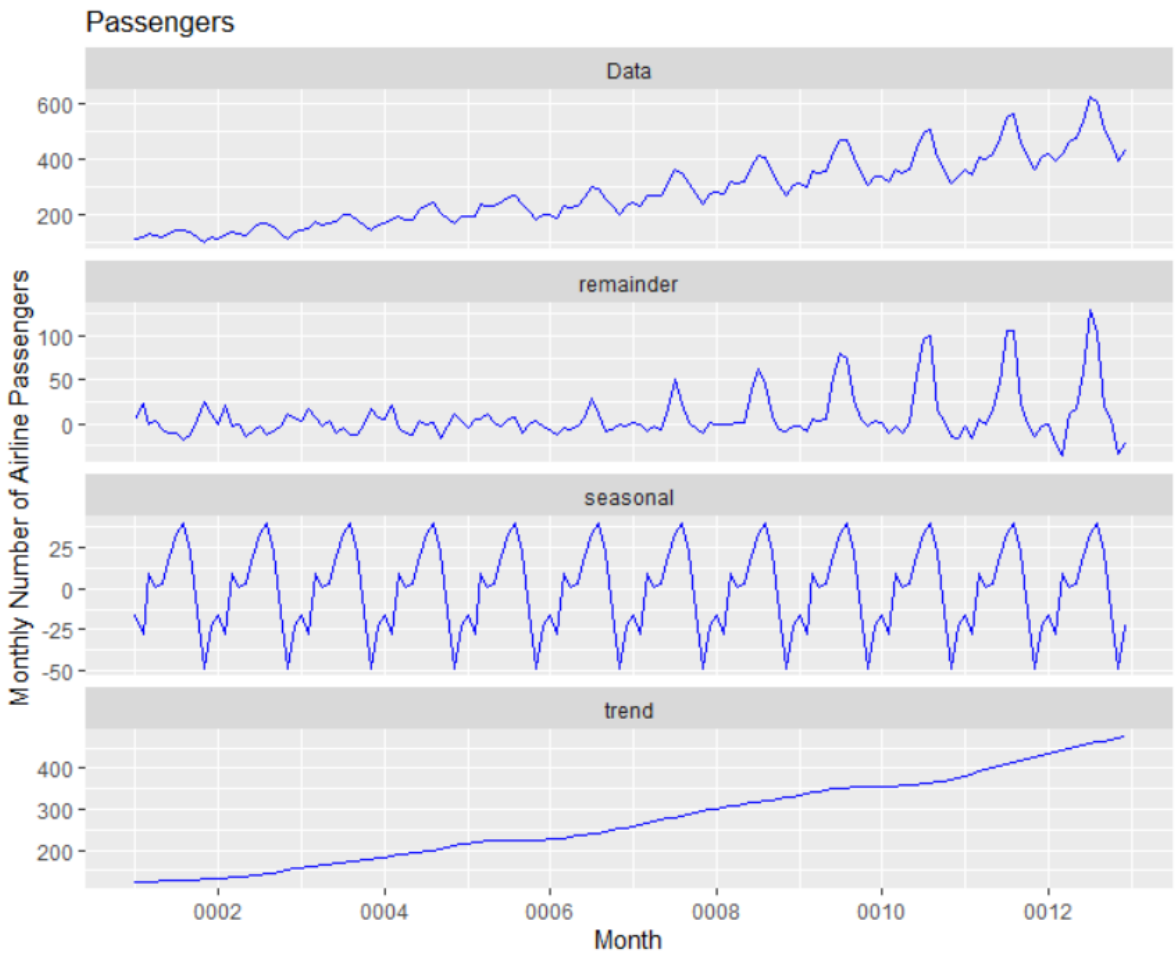
If you change the situation of the data slightly, the discrepancy with the measured values will be so large that it will not be a usable prediction model.






So, there is the idea of giving up **xgboost**. For example, **porohet** produces very good results, but lacks explanatory power. On the other hand, **xgboost** has explanatory power and is the best predictor among **non-deep learning** models, so it would be a shame to throw it away. So the idea is to use the best parts of **xgboost** in combination with other models as a compromise.

Fortunately, time series data can be used to isolate trends.

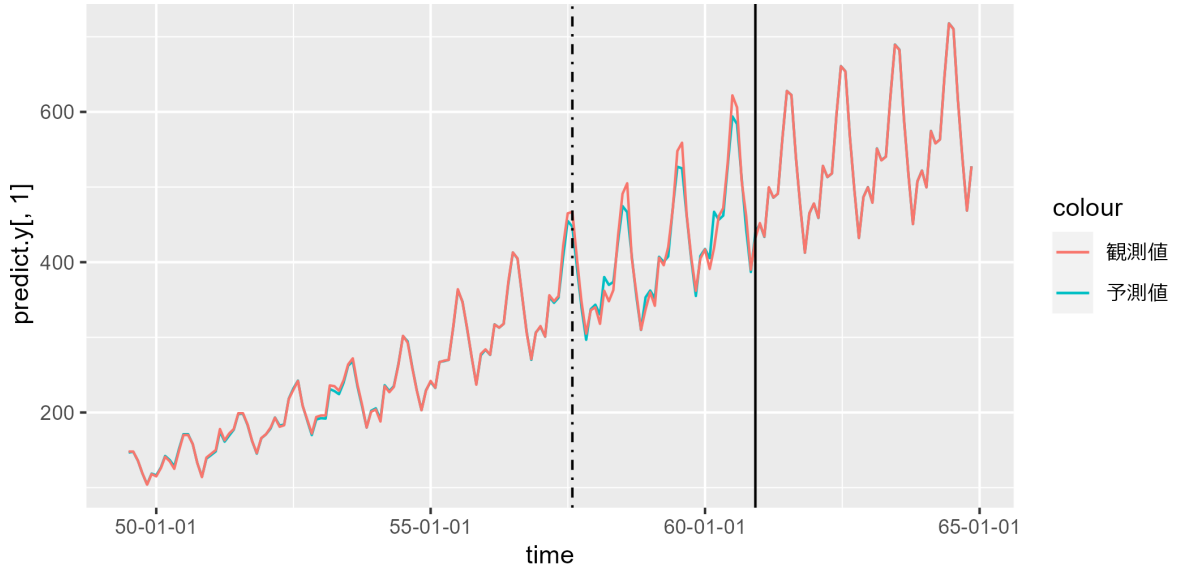



The data can be decomposed into **trend + seasonal + remainder**. If we let **xgboost** take care of the part without **trend**, and model **trend** with **ARIMA**, we can build a good model. However, depending on the data, **xgboost** may have weaknesses for repeated cycles.

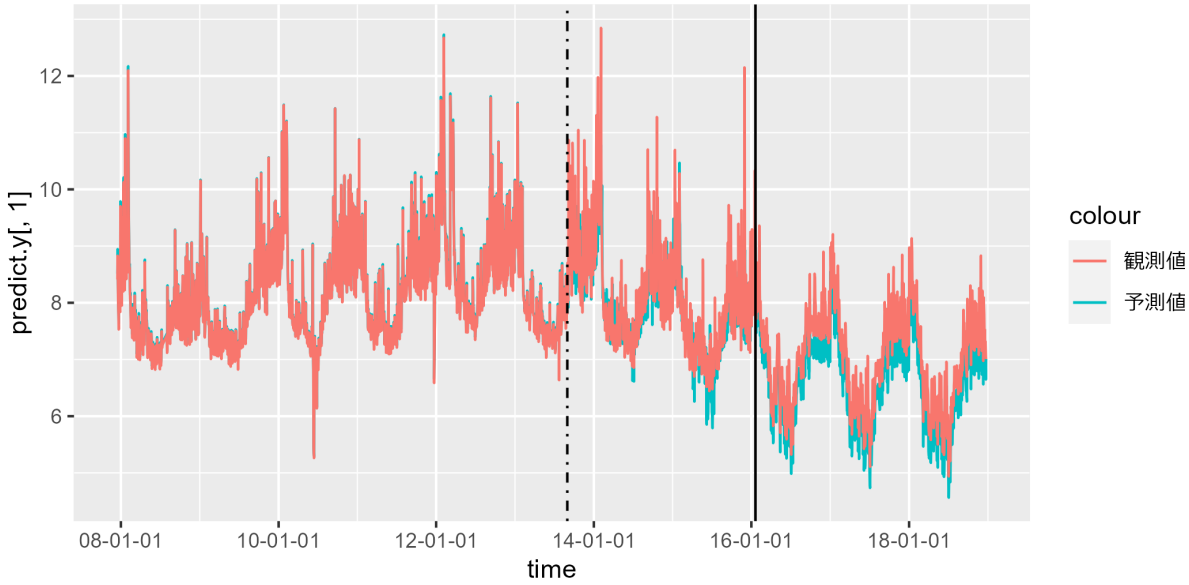
Therefore, I think it is possible to deal with this problem by adding the sin and cos terms of the Fourier expansion of the periodic component to the explanatory variables.

$$y_t = a + \sum_{k=1}^{K} \left[\alpha_k \sin(2\pi kt/m) + \beta_k \cos(2\pi kt/m)\right]$$

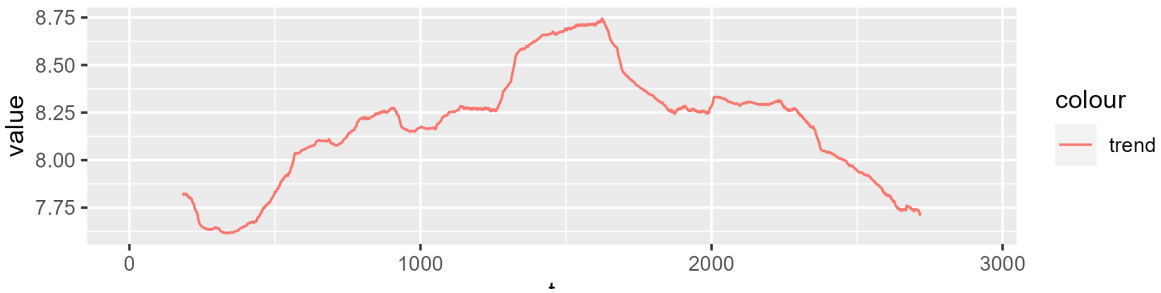
However, it is necessary to limit it to the day before the sin and cos terms are available. The reason is that the sin and cos terms on the day you want to predict are essentially unknown values. Therefore, they will be missing as explanatory variables on the day of the event, and we will have to fill them in by predicting. So, my idea is to use the previous day's sin and cos terms as they are to make a provisional forecast. This part of the explanatory variables can be refreshed each time the prediction is made, since the sin and cos terms can be re-calculated as the prediction progresses, thus eliminating the problem of having only the copied sin and cos terms of the previous day. However, it is possible to reinforce this problem. As a result of decomposing the time series data, we get seasonal, which is generally a simple repetition, and by using this simple repetition data, we can modify the explanatory variables to be more reasonable.



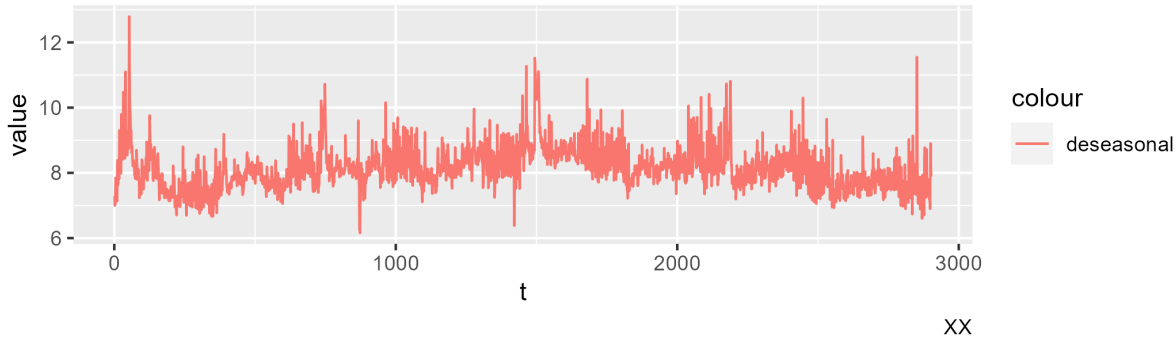

It seems that the prediction is reasonable even outside the data, i.e., in the extended future. Next, let's experiment with example_wp_log_peyton_manning.csv, which shows the results of extended prediction for 3 years (1096 steps).



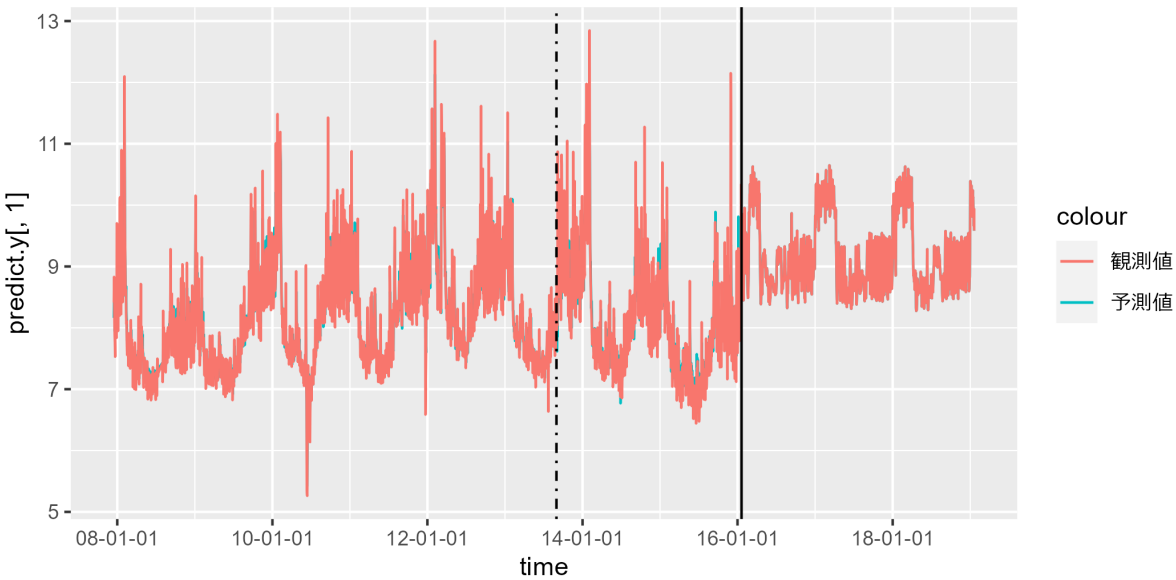
The trend component of this data is



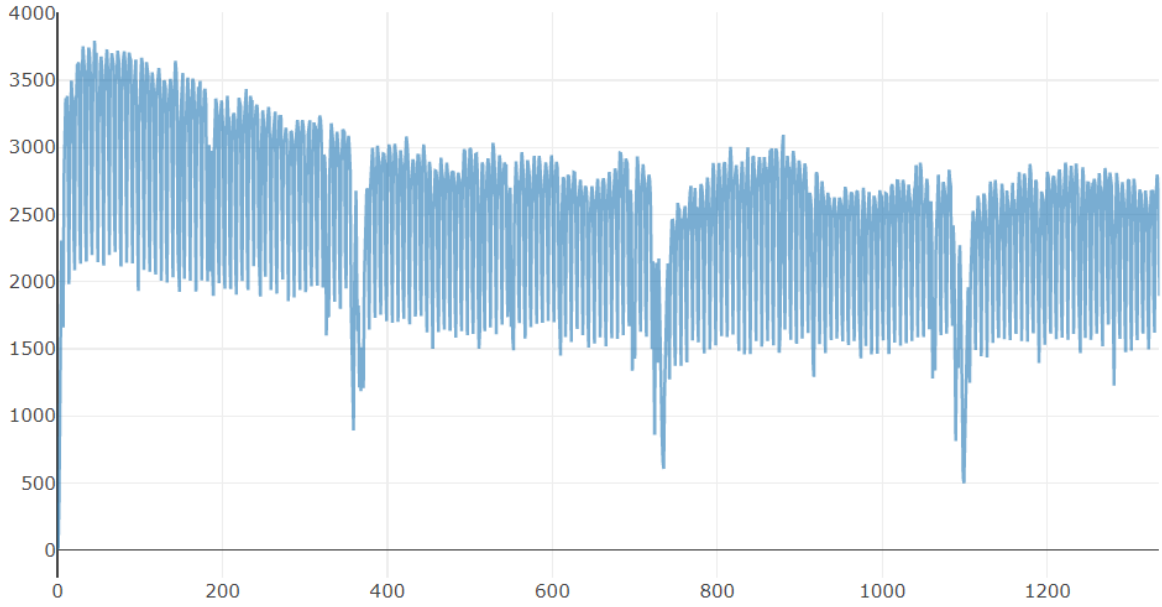

The downward trend shows that the forecast model follows the trend. The data for the component contributed by **xgtboost** is shown in the figure below, which shows that the forecasting model fully utilizes the superior capabilities of **xgboost**.



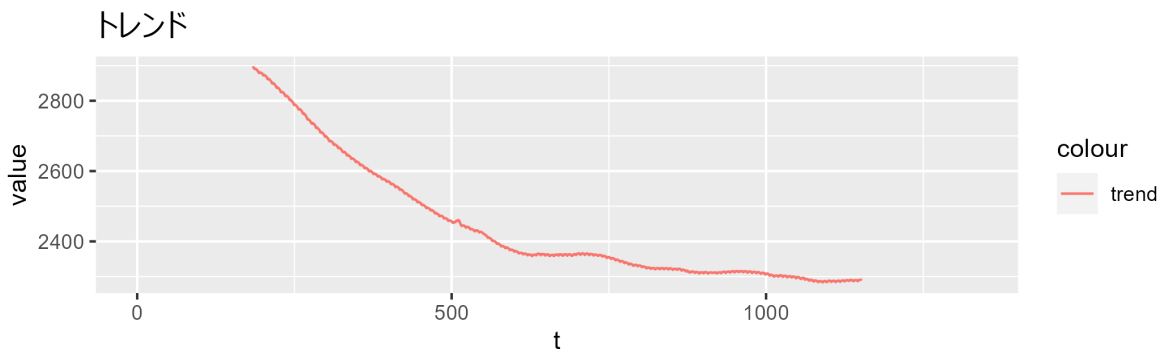

By the way, if we model this data using only **xgboost**, we get the following.



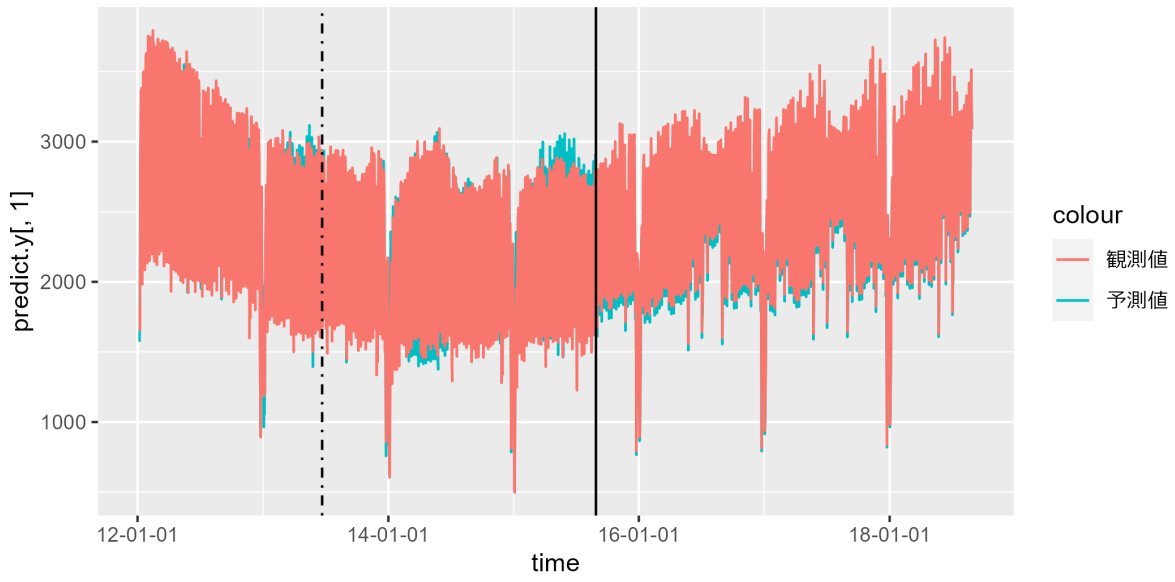

This suggests that the idea is producing reasonable results. Other data validation includes




his data appears to be right-side-up. In fact




The trend component is falling to the right. I tried to train this data by constraining the training interval to 40% of the total. The test interval is a very good predictor, but the extension to 3 years (1096 steps) shows a slight upward trend, but the characteristic increase and decrease seems to be a very good predictor. However, the characteristic increase/decrease looks like a very good prediction.



```
@inproceedings{y2021timeseriesxgboost,
  title={time series xgboost: time series xgboost},
  author={sanaxen},
  year={2021}
}
```